Designing Production-Ready Skills for AI Agents

How to extend AI agents with reusable, version-controlled capabilities

David Okwii

Lead Software Developer · Serve Digital

Africa's Talking · East Africa Hub

2026-05-07

A bit about me

- Lead Software Developer at **Serve Digital**
- Also run **dignited.com** — Uganda/Africa tech blog (since 2013) — where I've been building production AI agent pipelines
- Software developer, SEO practitioner, content creator
- Today: sharing what I've learned about **building skills**, not just using them

What this talk is (and isn't)

Is:

- Introduction to **agent skills** as a portable format
- Concrete examples from a real production pipeline
- Design principles + common mistakes
- A live demo

Isn't:

- "How LLMs work under the hood" (different talk)
- A pitch for one specific tool — skills work across many agents
- Advanced multi-agent architectures

Quick audience check

- How many of you have used **ChatGPT, Claude, or Gemini**?
- How many have used **Cursor, Claude Code, or GitHub Copilot** for actual coding?
- How many have built something that calls an LLM API?
- How many know what an **AI agent** is?

The agent moment we're in

> "AI agents" went from research demo to mainstream tool in ~18 months.

You can now ask a tool to:

- Read your codebase, plan a refactor, open a PR
- Triage incoming emails, draft replies
- Watch logs, alert on anomalies, suggest fixes
- **Run news editorial pipelines** (we'll see one)

But agents have a problem you might recognise.

The problem with off-the-shelf agents

A generic agent doesn't know:

- **Your company's** review process
- **Your codebase's** conventions
- **Your team's** voice and tone
- **Your industry's** specific quirks

You either repeat the same context every conversation, OR you accept generic output.

This is where skills come in.

What is an AI agent? (the short version)

An **agent** is an LLM that can:

1. **Receive a goal** ("draft this story for our blog")
2. **Plan steps** to achieve it
3. **Use tools** (run code, fetch URLs, edit files)
4. **Loop** — observe results, adjust, continue
5. **Report back** when done

Think: a junior developer with infinite patience and shaky judgement.

What is an agent skill?

> **A how-to guide for an agent.**

Like a recipe. Like an IKEA instruction sheet. Like a runbook.

A folder with a `SKILL.md` file that says:

- **What** the task is
- **When** to do it
- The **steps** in order — for the agent, not a human

The agent reads it like you'd read a cookbook: skim the title, decide if it's relevant, follow the steps.

What skills are NOT

❌ A **chat prompt** — those are one-off, not reusable

❌ A **system message** — those load on every conversation, even when irrelevant

❌ An **API call** — that's the engine; skills are the manual

❌ A **Wikipedia article** — those are reference, not action

❌ **Magic** — they're just Markdown files in a folder

✅ A **how-to guide for a task**. Read on demand. Version-controlled. Reusable across agents.

The format in one screen

```
my-skill/
├── SKILL.md          ← Required: metadata + instructions
├── scripts/          ← Optional: executable helpers
├── references/       ← Optional: detailed docs
└── assets/           ← Optional: templates, data
```

That's the whole format.

Open standard, originally from Anthropic, now supported by **30+ agent products** (Claude Code, Cursor, Gemini CLI, OpenCode, Goose, GitHub Copilot, …).

Source: agentskills.io

Agent vs skill — the analogy

	Agent	Skill
What it is	A worker	A **playbook** the worker reads
Lifespan	Per-session	Persistent on disk
Knows	General LLM stuff	Your specific task
Number per system	Usually 1	Usually many
Reusable across products?	No (vendor-locked)	**Yes** (open format)

> **Agents do work. Skills tell agents how YOUR work gets done.**

Why skills matter — three reasons

1. Capture institutional knowledge

Your "how we do X" lives in a file, not in someone's head.

2. Repeatability

Same task, same procedure, every time.

3. Cross-product portability

Write once, use in any agent that supports the format. No lock-in.

How agent skills work

Progressive disclosure — the skill loads in three stages:

```
Stage 1: DISCOVERY
  At startup, agent reads only `name` + `description` of every skill.
  ~100 tokens per skill.
  Just enough to know "this might be relevant".

Stage 2: ACTIVATION
  When a task matches a skill's description, agent reads the
  full SKILL.md body. ~500 to 5000 tokens.

Stage 3: EXECUTION
  Agent follows the instructions. Loads scripts/ or references/
  files only when needed.
```

Result: an agent can have 100 skills available with a tiny context cost.

Anatomy of a skill — the folder

```
african-tech-digest/
├── SKILL.md
│   ├── --- yaml frontmatter ---
│   │     name: african-tech-digest
│   │     description: Scan curated tech blogs and...
│   │   --- yaml frontmatter ---
│   └── (markdown instructions)
│
├── scripts/
│   └── fetch_feeds.py      # Stdlib Python feed fetcher
│
└── references/
    └── blogs.md            # 24 publication source list
```

A real skill from my dignited pipeline. We'll come back to this.

SKILL.md — the minimal example

```
---
name: pdf-processing
description: Extract PDF text, fill forms, merge files.
  Use when handling PDFs.
---

# PDF processing

To extract text:
1. Read the PDF file
2. Run `scripts/extract.py <file>`
3. Return the text

To fill a form:
1. Identify form fields
2. ...
```

That's it. Two required fields, a few lines of instructions, and you have a skill.

SKILL.md frontmatter spec

Field	Required	Constraint
`name`	**Yes**	1-64 chars, lowercase a-z + hyphens, must match folder name
`description`	**Yes**	1-1024 chars. **What it does AND when to use it.**
`license`	No	License name or reference
`compatibility`	No	Environment requirements (e.g. "Requires Python 3.14+")
`metadata`	No	Arbitrary key-value (author, version, etc.)
`allowed-tools`	No	Pre-approved tools (experimental)

The `description` is the **most important field** — it's how the agent decides whether to invoke your skill.

Bundled resources

The optional folders give skills more capability:

`scripts/` — Executable helpers

- Python, Bash, JavaScript
- Self-contained, with helpful error messages
- Loaded only when needed

`references/` — Domain documentation

- `REFERENCE.md`, `FORMS.md`, etc.
- Keep each file focused — agents load on demand

`assets/` — Static resources

- Templates, schemas, lookup tables
- Anything the agent uses but doesn't read into context

A real skill: `african-tech-digest`

Used in my production pipeline. Excerpt:

```
---
name: african-tech-digest
description: Scan a curated list of African tech blogs and
  return a ranked digest of the most interesting stories.
  Use whenever the user asks about "what's happening in
  African tech", a "weekly tech digest", "top tech stories
  from Africa/Nigeria/Kenya/SA"...
---

# African Tech Digest

## Ranking criteria
1. Filter out entries where noise_score >= 4 (...)
2. Multi-source clusters are top-story candidates
3. For single-source: funding, M&A, regulatory shifts, ...
```

Bundled: `scripts/fetch_feeds.py` (RSS fetcher), `references/blogs.md` (24 sources).

Design principles — what good skills look like

The next 5 slides walk through principles I've found matter most.

Source: Addy Osmani's *"Design Principles for Writing Good Agent Skills"* (with my own production lessons added).

Principle 1: Process over prose

> **Workflows are agent-actionable. Essays are not.**

❌ "PDFs can be processed in many ways. The most common approach is to extract text using a library like..."

✅

```
1. Run `scripts/extract.py <file>` — outputs JSON.
2. If status="encrypted", ask user for password.
3. Else, parse the `text` field.
4. Return text or report failure.
```

Skills should read like a **runbook**, not a wiki page.

Principle 2: Verification as non-negotiable

> Every workflow must terminate in **concrete evidence**.

Not enough:

- "Looks right"
- "Tests pass" *(passing tests are evidence, not proof)*
- "I think this is correct"

Required:

- "Compiled successfully + ran the build + here's the output"
- "All 12 assertions passed against today's data"
- "Posted as draft + here's the wp-admin link to verify"

Make 'produce evidence' the hard exit step.

Principle 3: Scope discipline

> **Touch only what you're asked to touch.**

Without this, agents:

- "While I'm here, let me modernise this old code"
- "I noticed an unrelated bug, fixing it too"
- "Let me also update these other files"

The skill must explicitly say:

> *"Don't refactor surrounding code. Don't fix unrelated bugs. Don't touch files outside the specified scope."*

Otherwise scope creep is the default.

Principle 4: Bounded complexity

> Keep individual skills **focused**.

Bad: One mega-skill called `do-everything` with 50 different workflows inside.

Good: Many focused skills.

```
.claude/skills/
├── african-tech-digest/      # Daily news scanning
├── dignited-news-rewrite/    # Voice transfer + WP publish
└── humanizer/                # AI-tell filter pass
```

Complex tasks **chain skills** rather than collapse into one.

Principle 5: Anti-rationalization

> Document common excuses paired with rebuttals.

Without this, agents generate plausible justifications for skipping work:

> *"Tests pass, so we can ship."* → Passing tests are evidence, not proof. **Did you also build it? Did you run it manually? Did you check the logs?**

> *"This file looks fine."* → Looks fine is not the test. **Did you run `<the actual check>`?**

The agent will rationalise its way past missing steps unless you preempt it.

Anti-patterns — what NOT to do

❌ **Long reference essays** — agents skim, they don't study

❌ **Vague success criteria** ("looks good", "should work")

❌ **Hidden invisible work** — skipping specs, tests, reviews

❌ **Loading everything upfront** — kills context, slows runs

❌ **Crossing trust boundaries silently** — touching production, sending messages, deleting data without a checkpoint

❌ **One mega-skill for everything** — split it

❌ **Names that don't match the action** — `description` should pattern-match what users actually say

A common anti-pattern in detail

❌ The vague description:

```
name: helper
description: Helps with various tasks.
```

The agent has no idea when to invoke this. It probably won't.

✅ The triggerable description:

```
name: pdf-processing
description: Extract PDF text, fill forms, merge files.
  Use when working with PDF documents or when the user
  mentions PDFs, forms, or document extraction.
```

Specific verbs, named scenarios, keywords users actually type.

Demo time — let's send an SMS from this room

We'll build the full **discovery** → **activation** → **execution** loop in front of you. A real SMS will land on a real phone in this room.

What we'll watch:

1. The skill **source** — `SKILL.md` and a small Python script (no SDK, no magic)
2. An agent **discover** the skill from its description alone
3. The agent **activate** it (read the full file)
4. The agent **execute** — call the Africa's Talking SMS API
5. **A volunteer's phone buzzes** with the SMS — verifiable proof

Why this skill, why this venue: AT host this event. Showing their API integrated with an AI agent feels like the right kind of nod.

Demo: setup (1 min)

What's needed:

- Claude Code (or any skill-compatible agent)
- A folder at `~/.claude/skills/africastalking-sms/` with:
  - `SKILL.md` — the instructions
  - `scripts/send_sms.py` — the executor (Python stdlib only, no SDK)
- Three env vars in the shell: `AT_USERNAME`, `AT_API_KEY`, `AT_SENDER_ID`

What we explicitly DON'T do:

❌ Embed the API key in `SKILL.md` (it would leak via git the moment someone commits)

❌ Hard-code the phone number (the agent gets it from the user's natural-language prompt)

These are themselves teaching points for the talk.

Demo: the SKILL.md (90s)

```
---
name: africastalking-sms
description: Send an SMS via the Africa's Talking SMS API.
  Use whenever the user wants to send a text message, alert,
  OTP, notification, "send SMS to <number>", or any phrasing
  that involves delivering a short message to a mobile phone
  in Africa.
license: MIT
compatibility: Requires Python 3 and AT_USERNAME +
  AT_API_KEY + AT_SENDER_ID env vars.
---

# Africa's Talking SMS

## When to use
Trigger when the user asks to send an SMS, OTP, alert, ...

## Required environment variables
| Variable       | Example       |
| AT_USERNAME    | `odukar`      |
| AT_API_KEY     | `atsk_xxxxx…` |
| AT_SENDER_ID   | `DIGNITED`    |

Sender ID is **required** — AT (and Uganda's regulator)
won't deliver bulk SMS without a registered one.

Never paste the API key into a SKILL.md or commit it to git.
```

Demo: live (3 min)

In Claude Code, I type:

> *"Send an SMS to +256 <volunteer-number> saying 'Hello from a Claude skill demo at Africa's Talking Kampala'"*

Watch what happens:

1. **Discovery** — Claude scans available skills. `africastalking-sms` matches.
2. **Activation** — Claude reads `SKILL.md`, sees the workflow.
3. **Execution** — Claude runs:

```
python3 ~/.claude/skills/africastalking-sms/scripts/send_sms.py \
  --to "+256..." \
  --message "Hello from a Claude skill demo at Africa's Talking Kampala"
```

4. **Result** — JSON response with `accepted: 1`, message ID, cost
5. **Volunteer holds up phone with the SMS visible.** 🎉

And one bigger production example (briefly)

Also using skills daily: my dignited.com editorial pipeline.

Three skills working together:

- `african-tech-digest` — scans 24 publishers, ranks stories
- `dignited-news-rewrite` — drafts a story in dignited voice, posts as WP draft
- `humanizer` — final-pass AI-tell filter

Fires every morning at 11:00 EAT. I read the digest, reply with story numbers in chat, drafts appear in wp-admin within 3 minutes.

Skills compose. A complex pipeline is many small skills, not one giant one.

What you'd build for your own use case

A few ideas, riffing on AT's APIs:

- **`at-airtime-topup`** — *"Send 5,000 UGX of airtime to +256…"* via AT Airtime API
- **`at-otp-sender`** — One-time password generator + SMS in one skill, with `--length 6 --validity 5min`
- **`at-bulk-broadcast`** — Read a CSV of customers, send personalised SMS to each, return a delivery report
- **`at-voice-call-trigger`** — Place a voice call with a recorded message via AT Voice API
- **`at-payment-collector`** — Initiate mobile-money collection via AT Payments

Or anything in your own life:

- `commit-message-writer` from `git diff`
- `student-grade-summary` from a CSV

Browse before you build — skill marketplaces

Most common tasks already have skills written for them. Check these first:

Where	What it has
skills.sh	Open agent skills directory. Install with `npx skills add <name>`
clawhub.ai	Claude / agent skills marketplace
claudemarketplaces.com/skills	4,200+ Claude Code skills, browseable by category

Install with one command:

```
# install Anthropic's official skill-creator into your local Claude
npx skills add anthropics/skills@skill-creator
```

Open-source skill collections on GitHub

Repo	What
`anthropics/skills`	Official Anthropic skills (incl. skill-creator)
`addyosmani/agent-skills`	Addy Osmani's curated collection
`agentskills/agentskills`	The open standard reference repo
`vercel-labs/skills`	The `npx skills` CLI itself

Read other people's skills first. The format is so simple that the easiest way to learn is to copy a working skill, edit it, see what happens.

Where to learn more

- **Specification**: agentskills.io/specification
- **Quickstart**: agentskills.io/skill-creation/quickstart
- **Addy Osmani's design principles**: addyosmani.com/blog/agent-skills
- **Anthropic's docs**: code.claude.com/docs/en/skills
- **Discord (active community)**: discord.gg/MKPE9g8aUy
- **My production pipeline**: dignited.com source on request

Practical next steps

If you're starting today:

1. **Pick one repetitive task** you do at work / school
2. **Write down the steps** as bullet points (no prose)
3. **Save it as `SKILL.md`** with `name` + `description` frontmatter
4. **Drop it into Claude Code, Cursor, or any compatible agent**
5. **Iterate** based on what the agent gets right/wrong

Don't try to write the perfect skill on day one. Ship a rough one, watch what fails, fix.

Questions?

Find me:

- 🌐 dignited.com
- 🐦 @oquidave
- 📧 oquidave@gmail.com

Slides + demo files: *(URL when published)*

Thanks Africa's Talking, thanks Kampala. 🇺🇬